

Module 2: Transport Layer and Network Layer



Module 2: Transport Layer and Network Layer

Transport Layer

- Services and Protocols
- UDP and TCP

Hands-on

- Sockets Introduction
- Elementary TCP Sockets
- TCP Client/Server Example
- I/O Multiplexing
 - select()
 - poll()
- Elementary UDP Sockets
- Advanced I/O Functions

Network Layer

- Introduction
- Network-layer Protocols
- Unicast Routing
- Multicast Routing
- Multicasting Basics
- Intra-domain Routing
- Inter-domain Routing
- Next Generation IP (IPv6)
- Quality of Service (QoS)

Hands-on

- Linux Routing Table
- Routing Caches
- Routing Cache Implementation
- Adding Routes using ip

Unicast Routing – Basic Idea

The network layer delivers a datagram from a source to its destination.

Types of Delivery

- **Unicast** – one-to-one delivery
- **Multicast** – one-to-many delivery
- **Broadcast** – one-to-all delivery

This section concentrates on **unicast routing**.

Internet Routing

Because the Internet contains a huge number of routers and hosts, unicast routing uses **hierarchical routing**.

Routing is performed in several steps using different routing algorithms and protocols.

General Idea of Unicast Routing

A packet is routed **hop by hop** using forwarding tables.

- Source host does not need a forwarding table.
- It sends the packet to its default router.
- Destination host does not need a forwarding table.
- It receives the packet from its default router.
- Routers connecting networks require forwarding tables.

Thus routing can be viewed as:

Source Router → Intermediate Routers → Destination Router

The key question is:

Which route should the packet take?

Several possible routes may exist between source and destination.

Internet as a Weighted Graph

An Internet can be modeled as a graph.

- Router $\rightarrow$ node
- Network between routers $\rightarrow$ edge
- Edge $\rightarrow$ associated with a cost

Weighted Graph

Each edge has a cost representing some characteristic of the route. The interpretation of cost depends on the routing protocol.

If two nodes have no connecting edge:

$$\text{Cost} = \infty$$

Analogy:

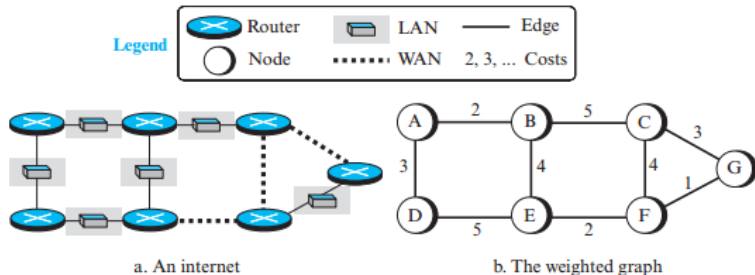
Cities $\rightarrow$ nodes,

Roads $\rightarrow$ edges,

Distance $\rightarrow$ cost

Figure 4.56 – Internet as a Weighted Graph

Figure 4.56 *An internet and its graphical representation*



Least-Cost Routing and Least-Cost Trees

Least-Cost Routing

The best route is the route whose total cost is the minimum among all possible routes.

Example from Figure 4.56: $A \rightarrow B \rightarrow E$
with total cost:



If an Internet contains N routers: $N(N-1)$
least-cost paths are needed if paths are considered from every router to every other router.

Least-Cost Tree

A tree rooted at a source router that spans the entire graph and contains the shortest path from the root to every other node.

Therefore, only N least-cost trees are needed.

Least-Cost Tree – Important Properties

Property 1 – Reverse Paths

The least-cost route from X to Y in X 's tree is the reverse of the least-cost route from Y to X in Y 's tree.

The cost is the same in both directions.

Example:

$$A \rightarrow B \rightarrow E \rightarrow F$$

$$F \rightarrow E \rightarrow B \rightarrow A$$

Both have cost 8.

Property 2 – Combining Trees

A route can be constructed by following a path in one tree and then continuing using another node's tree.

Example: $A \rightarrow B \rightarrow E$
then

$$E \rightarrow F \rightarrow G$$

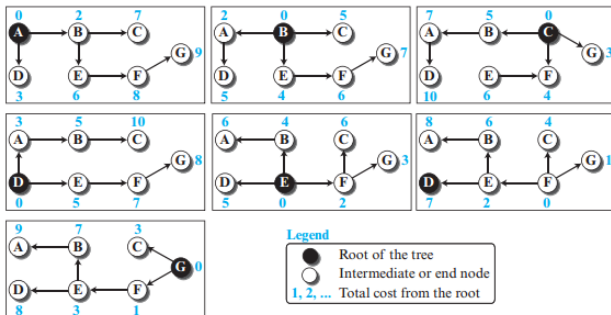
Therefore:

$$A \rightarrow B \rightarrow E \rightarrow F \rightarrow G$$

$$\text{Cost} = 9 = 6 + 3$$

Figure 4.57 – Least-Cost Trees

Figure 4.57 Least-cost trees for nodes in the internet of Figure 4.56



Routing Algorithms

Routing algorithms differ mainly in:

- How they interpret the cost
- How they construct least-cost trees
- How routing information is exchanged

Three major routing algorithms discussed:

Distance-Vector Routing

Uses neighboring routers and the Bellman–Ford equation.

Link-State Routing

Builds a complete link-state database and uses Dijkstra's algorithm.

Path-Vector Routing

Selects paths according to source-defined policies rather than least-cost alone.

Distance-Vector Routing

Distance-vector (DV) routing uses the least-cost goal.

Basic process:

- 1 Each router initially creates a rudimentary least-cost tree.
- 2 It knows the cost to its immediate neighbors.
- 3 It creates a distance vector.
- 4 The vector is sent to immediate neighbors.
- 5 Neighbors update their vectors.
- 6 Updated vectors are repeatedly exchanged.

Main Idea

A router continuously tells its neighbors what it knows about the whole Internet, although its knowledge may initially be incomplete.

Two important concepts:

Bellman–Ford Equation

Distance Vector

Bellman-Ford Equation

The general equation for the least cost from x to y is:

$$D_{xy} = \min\{c_{xa} + D_{ay}, c_{xb} + D_{by}, c_{xc} + D_{cy}, \dots\}$$

where:

- D_{ij} = shortest distance from node i to node j
- c_{ij} = cost between nodes i and j

For updating an existing least cost through an intermediary z :

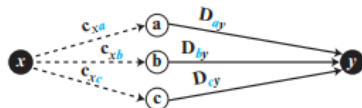
$$D_{xy} = \min\{D_{xy}, c_{xz} + D_{zy}\}$$

Meaning

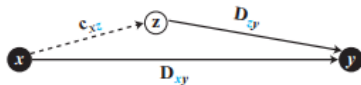
A new least-cost path is constructed from previously established least-cost paths.

Figure 4.58 – Bellman–Ford Equation

Figure 4.58 Graphical idea behind Bellman-Ford equation



a. General case with three intermediate nodes



b. Updating a path with a new route

Distance Vector

A least-cost tree contains paths from the root to all destinations.

A distance vector represents the same information as a **one-dimensional array**.

- Vector name identifies the root.
- Index identifies a destination.
- Value identifies the least cost to that destination.

Important Difference

A least-cost tree shows the actual paths. A distance vector stores only the **least costs**, not the paths.

When a router starts:

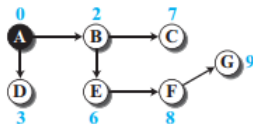
$$D[\text{self}] = 0$$

$$D[\text{neighbor}] = \text{direct cost}$$

$$D[\text{unknown}] = \infty$$

Figure 4.59 – Distance Vector

Figure 4.59 *The distance vector corresponding to a tree*



a. Tree for node A

A	
A	0
B	2
C	7
D	3
E	6
F	8
G	9

b. Distance vector for node A

Distance-Vector Updates

Initially, each router knows only its neighbors.

- It sends its vector to all immediate neighbors.
- A neighbor receives the vector.
- The neighbor applies the Bellman–Ford equation.
- All entries are checked for improvement.
- If the vector changes, the updated vector is sent again.

The process is:

Initialize → Exchange → Update → Exchange → ...

Eventually the vectors stabilize and represent the least costs.

Important

The process is **asynchronous**; routers do not necessarily create or exchange vectors simultaneously.

Figure 4.61 – Distance-Vector Update Process

Figure 4.61 Updating distance vectors

New B		Old B		A	
A	2	A	2	A	0
B	0	B	0	B	2
C	5	C	5	C	∞
D	5	D	∞	D	3
E	4	E	4	E	∞
F	∞	F	∞	F	∞
G	∞	G	∞	G	∞

$B[j] = \min(B[j], 2 + A[j])$

a. First event: B receives a copy of A's vector.

New B		Old B		E	
A	2	A	2	A	∞
B	0	B	0	B	4
C	5	C	5	C	∞
D	5	D	5	D	5
E	4	E	4	E	0
F	6	F	∞	F	2
G	∞	G	∞	G	∞

$B[j] = \min(B[j], 4 + E[j])$

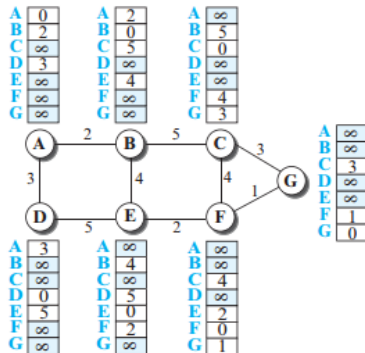
b. Second event: B receives a copy of E's vector.

Note:

$X[j]$: the whole vector

Figure 4.60 – Initial Distance Vectors

Figure 4.60 *The first distance vector for an internet*



Distance-Vector Routing Algorithm – Table 4.4

Initialization

```
Distance_Vector_Routing ( )
{
    D[myself] = 0

    for (y = 1 to N)
    {
        if (y is a neighbor)
            D[y] = c[myself][y]
        else
            D[y] = infinity
    }

    send vector {D[1], D[2], ..., D[N]}
    to all neighbors
}
```

Update Process

```
repeat (forever)
{
    wait (for a vector Dw from
    a neighbor w or any change
    in the link)

    for (y = 1 to N)
    {
        D[y] = min[D[y],
        (c[myself][w] + Dw[y])]
    }

    if (any change in the vector)
        send vector {D[1], ..., D[N]}
        to all neighbors
}
}
```

Count-to-Infinity and Two-Node Loop

Count-to-Infinity

Good news, such as a decrease in cost, propagates quickly.

Bad news, such as a broken link, can propagate slowly.

If a link becomes unavailable:

$$\text{Cost} \rightarrow \infty$$

Routers may continue believing that a route exists through another router.

Two-Node Loop

After a link to destination X fails:

$$A \rightarrow B \rightarrow X$$

may be believed by A , while:

$$B \rightarrow A \rightarrow X$$

may be believed by B .

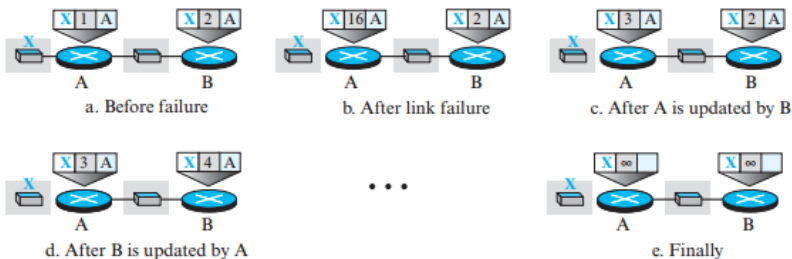
Packets can therefore bounce:

$$A \rightarrow B \rightarrow A \rightarrow B \rightarrow \dots$$

until the metric eventually reaches infinity.

Figure 4.62 – Two-Node Loop

Figure 4.62 Two-node instability



Split Horizon and Poisoned Reverse

Split Horizon

A router does not advertise a route back through the interface from which it learned that route.

Example:

If B learned the route to X from A :

$B \nrightarrow A$: route to X

This prevents the information from being sent back to its source.

Poisoned Reverse

The router advertises the route back to its source, but with:

$$\text{Cost} = \infty$$

This explicitly means:

*"Do not use this route;
I learned it from you."*

Purpose

These techniques help reduce two-node instability.

Three-Node Instability

Split horizon combined with poisoned reverse can avoid **two-node instability**.

However:

Three-node instability is not completely eliminated.

A routing loop can involve:



The routing information may circulate among the three routers before the failure is recognized.

Limitation of Distance Vector

Distance-vector routing depends on information received from neighbors, so incorrect information can propagate through the domain.

Link-State Routing

Link-state (LS) routing directly follows the least-cost-tree approach.

Link State

The cost associated with an edge represents the state of the link.

- Low cost → preferred
- High cost → less preferred
- Infinite cost → link unavailable

Each router needs a complete map of the network.

Link-State Database (LSDB)

The collection of the states of all links in the network.

Each router maintains a copy of the LSDB and uses it to create its own least-cost tree.

LSDB, Flooding and LSP

Each router discovers its immediate neighbors and obtains:

- Neighbor identity
- Cost of the link

These are placed into a:

Link-State Packet (LSP)

The LSP is sent through each interface.

Flooding

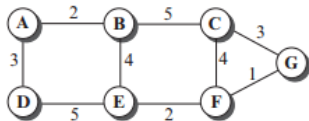
When a router receives an LSP:

- Older LSP → discard
- Newer LSP → keep
- Forward the new LSP through other interfaces

Sequence numbers prevent old information from being accepted. Eventually every router obtains the same comprehensive LSDB.

Figure 4.63 – Link-State Database

Figure 4.63 *Example of a link-state database*



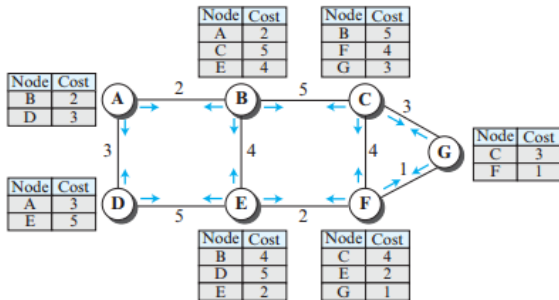
a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

b. Link state database

Figure 4.64 – Flooding of Link-State Packets

Figure 4.64 *LSPs created and sent out by each node to build LSDB*



Distance Vector vs. Link State

Distance Vector	Link State
Router tells neighbors what it knows about the whole network	Router tells the whole network what it knows about its neighbors
Uses distance vectors	Uses link-state packets
Uses Bellman–Ford equation	Uses Dijkstra's algorithm
Knowledge initially incomplete	Each router builds complete LSDB
Information exchanged with neighbors	LSPs are flooded through the area

Core Difference

DV distributes **distance information**; LS distributes **link information**.

Dijkstra's Algorithm – Least-Cost Tree

Dijkstra's algorithm creates a least-cost tree using the LSDB.

- 1 Select the source as the root.
- 2 Initialize costs using the LSDB.
- 3 Select the node with the smallest cost that is not yet in the tree.
- 4 Add the node to the tree.
- 5 Update costs of its neighbors.
- 6 Repeat until every node is included.

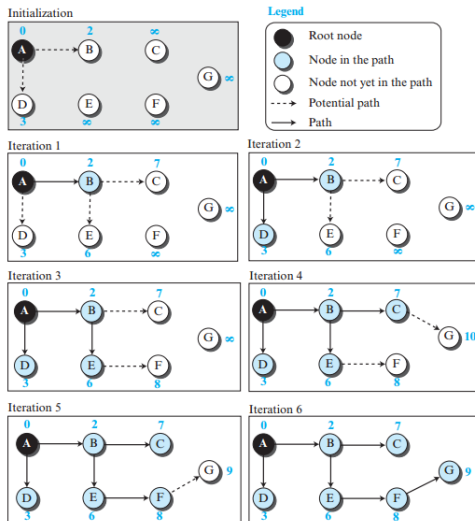
Core update:

$$D[x] = \min\{D[x], D[w] + c[w][x]\}$$

where w is the newly selected node.

Figure 4.65 – Dijkstra Least-Cost Tree Formation

Figure 4.65 Least-cost tree



Dijkstra's Algorithm – Table 4.5

Initialization

```
Dijkstra's Algorithm ( )
{
    Tree = {root}

    for (y = 1 to N)
    {
        if (y is the root)
            D[y] = 0
        else if (y is a neighbor)
            D[y] = c[root][y]
        else
            D[y] = infinity
    }
```

Tree Construction

```
repeat
{
    find a node w, with D[w] minimum
    among all nodes not in Tree

    Tree = Tree union {w}

    for (every node x, which is neighbor
        and not in Tree)
    {
        D[x] = min{D[x], (D[w] + c[w][x])}
    }

} until (all nodes included)
}
```

Key Idea

Select the closest node not yet in the tree, add it, update its neighbors, and repeat until all nodes are included.

Path-Vector Routing

Least-cost routing is not always sufficient.

An organization may want to:

- Avoid particular routers
- Avoid particular organizations
- Apply security policies
- Apply administrative policies
- Control the path followed by packets

Path-Vector Routing

The source selects a route according to its own policy rather than simply selecting the least-cost path.

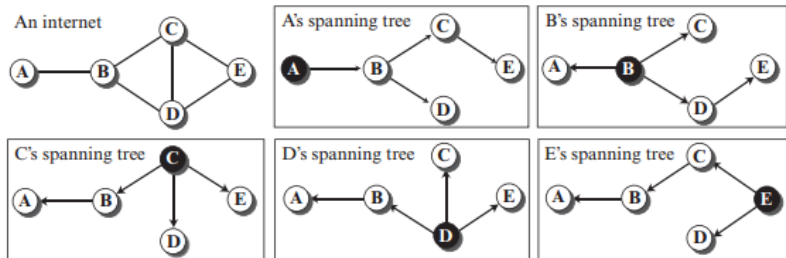
A source creates a **spanning tree** according to its policy.

Common policies include:

- Minimum number of nodes
- Avoiding selected nodes as intermediate nodes

Figure 4.66 – Spanning Trees in Path-Vector Routing

Figure 4.66 *Spanning trees in path-vector routing*



Path-Vector Algorithm

Path-vector routing is distributed and asynchronous.

A node initially creates paths based on information about its neighbors.

The update equation is:

$$Path(x, y) = best\{Path(x, y), [x + Path(v, y)]\}$$

The source chooses the best path according to its policy.

Loop Prevention

If $Path(v, y)$ already includes x , that path is discarded.

Thus a node does not select a path that returns to itself.

The final stabilized path vector can serve as the forwarding table.

Figure 4.67 – Initial Path Vectors

Figure 4.67 Path vectors made at booting time

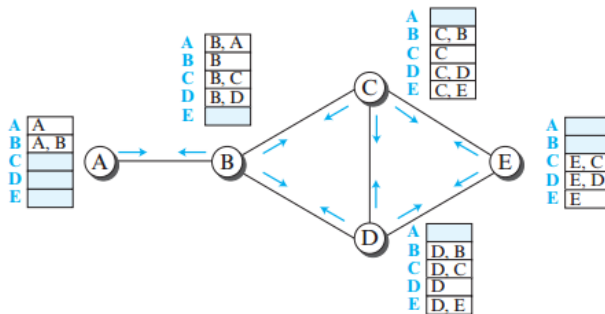
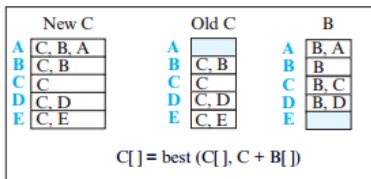
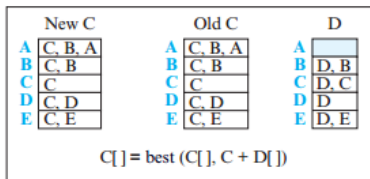


Figure 4.68 – Path-Vector Update

Figure 4.68 *Updating path vectors*



Event 1: C receives a copy of B's vector



Event 2: C receives a copy of D's vector

Note:

X []: vector X

Y: node Y

Path-Vector Routing Algorithm – Table 4.6

Initialization

```
Path_Vector_Routing ( )
{
    for (y = 1 to N)
    {
        if (y is myself)
            Path[y] = myself
        else if (y is a neighbor)
            Path[y] = myself + neighbor
        else
            Path[y] = empty
    }

    Send vector {Path[1], Path[2], ..., Path[N]}
    to all neighbors
}
```

Key Idea

The complete path is exchanged rather than only a distance. A path containing the current router is discarded to prevent loops.

Path Update

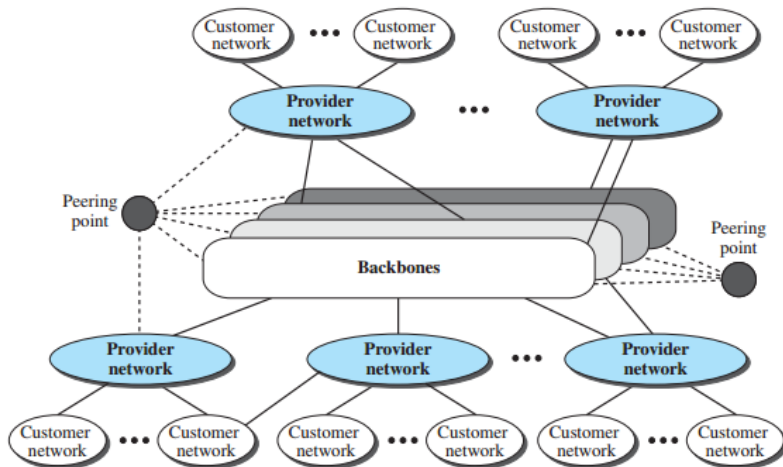
```
repeat (forever)
{
    wait (for a vector Pathw from a neighbor)

    for (y = 1 to N)
    {
        if (Pathw includes myself)
            discard the path
        else
            Path[y] = best
            {Path[y], (myself + Pathw[y])}
    }

    if ( change in the vector)
        Send vector
        {Path[1], ..., Path[N]}
        to all neighbors
    }
}
```

Figure 4.69 – Path-Vector Routing Example

Figure 4.69 *Internet structure*



Unicast Routing Protocols

A routing protocol is more than an algorithm.

It defines:

- Domain of operation
- Messages exchanged
- Communication between routers
- Interaction with other routing protocols

Three important Internet protocols:

Protocol	Algorithm	Domain
RIP	Distance Vector	Intra-AS
OSPF	Link State	Intra-AS
BGP4	Path Vector	Inter-AS

Memory Trick

$RIP \rightarrow DV, \quad OSPF \rightarrow LS, \quad BGP \rightarrow PV$

Internet Structure and Hierarchical Routing

The modern Internet contains:

- Multiple backbones
- Provider networks
- Customer networks
- Peering points

These entities can operate as **Internet Service Providers (ISPs)** at different levels.

Why Hierarchical Routing?

Scalability problem:

- Huge forwarding tables
- Longer searches
- Large update traffic

Administrative problem:

- Each organization needs control over its own routers, networks and policies.

Autonomous Systems and Hierarchical Routing

An Internet Service Provider can be treated as an:

Autonomous System (AS)

Each AS is assigned an Autonomous System Number (ASN).

Routing Hierarchy

Intra-AS Routing

- Also called intradomain routing or IGP
- RIP and OSPF

Inter-AS Routing

- Also called interdomain routing or EGP
- BGP

Three Types of AS

• Stub AS

One connection; traffic starts/ends there.

• Multihomed AS

Multiple connections; does not carry transit traffic.

• Transient AS

Multiple connections and carries transit traffic.

RIP – Routing Information Protocol

RIP is an **intradomain distance-vector routing protocol**.

Metric: Hop Count

RIP measures cost as the number of networks/subnets crossed.

Maximum cost = 15

16 = infinity / unreachable

RIP forwarding table:

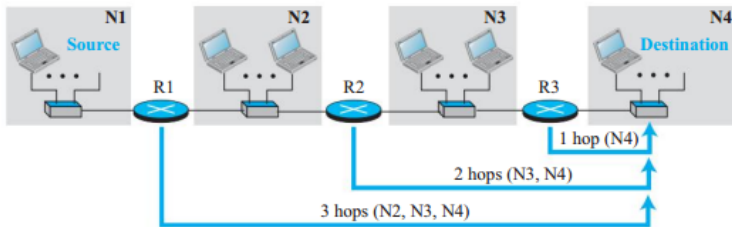
Destination Network	Next Router	Cost
Network	Next hop	Hop count

RIP runs using:

UDP port 520

Figure 4.70 – RIP

Figure 4.70 *Hop counts in RIP*



RIP Messages and Algorithm

RIP uses two major message types:

Request

Used by a router that has just started or needs route information. It may request:

- Specific routes
- All routes

Response / Update

Can be:

- **Solicited** – response to a request
- **Unsolicited** – periodic update or update after a change

RIP routers exchange their forwarding tables.

The receiving router:

- 1 Adds one hop to the received cost.
- 2 Sets next router to the sending router.
- 3 Adds new routes.
- 4 Accepts lower-cost routes.
- 5 Accepts a higher cost if the same next router advertised it.

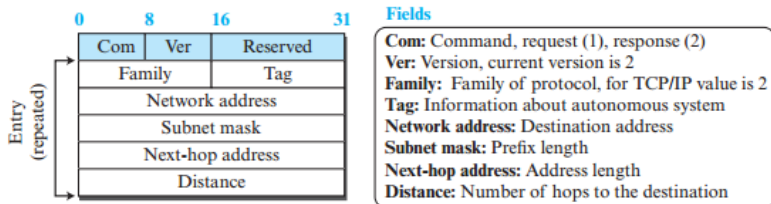
Figure 4.71 – RIP Messages

Figure 4.71 *Forwarding tables*

Forwarding table for R1			Forwarding table for R2			Forwarding table for R3		
Destination network	Next router	Cost in hops	Destination network	Next router	Cost in hops	Destination network	Next router	Cost in hops
N1	—	1	N1	R1	2	N1	R2	3
N2	—	1	N2	—	1	N2	R2	2
N3	R2	2	N3	—	1	N3	—	1
N4	R2	3	N4	R3	2	N4	—	1

Figure 4.72 – RIP Routing Process

Figure 4.72 *RIP message format*



RIP Timers and Performance

RIP uses three important timers.

Periodic Timer

- Randomly 25–35 s
- Controls regular updates

Expiration Timer

- 180 s
- Route becomes invalid
- Metric becomes 16

Garbage Collection

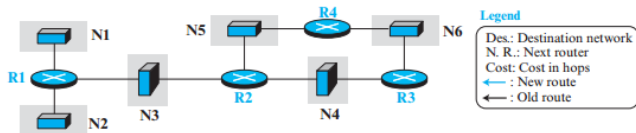
- 120 s
- Invalid route is eventually removed

Performance

- Simple local update messages
- Convergence can be slow in large domains
- Maximum 15-hop limit restricts domain size
- Split horizon and poisoned reverse reduce loops
- Failure/corruption can propagate through routers

Figure 4.73 – RIP Timers and Performance

Figure 4.73 Example of an autonomous system using RIP



R1			R2			R3			R4		
Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost
N1		1	N3		1	N4		1	N5		1
N2		1	N4		1	N6		1	N6		1
N3		1	N5		1						

Forwarding tables after all routers booted

New R1			Old R1			R2 Seen by R1		
Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost
N1		1	N1		1	N3	R2	2
N2		1	N2		1	N4	R2	2
N3		1	N3		1	N5	R2	2
N4	R2	2						
N5	R2	2						

New R3			Old R3			R2 Seen by R3		
Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost
N3	R2	2	N4		1	N3	R2	2
N4		1	N6		1	N4	R2	2
N5	R2	2				N5	R2	2
N6		1						

New R4			Old R4			R2 Seen by R4		
Des.	N. R.	Cost	Des.	N. R.	Cost	Des.	N. R.	Cost
N3	R2	2	N5		1	N3	R2	2
N4	R2	2	N6		1	N4	R2	2
N5		1				N5	R2	2
N6		1						

Changes in the forwarding tables of R1, R3, and R4 after they receive a copy of R2's table

OSPF – Open Shortest Path First

OSPF is an **intradomain link-state routing protocol**.

Metric

OSPF can assign link weights based on:

- Throughput
- Round-trip time
- Reliability
- Hop count
- Service type

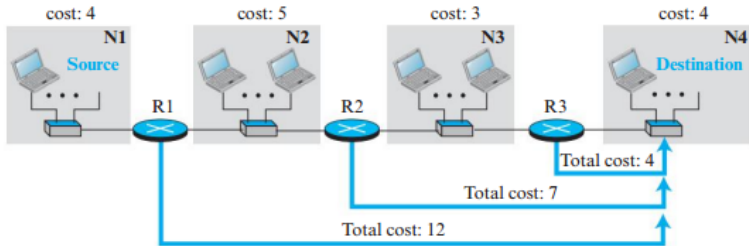
Each router:

LSDB → Dijkstra → Shortest-Path Tree → Forwarding Table

Unlike RIP, OSPF is designed for both small and large autonomous systems.

Figure 4.74 – OSPF

Figure 4.74 *Metric in OSPF*



OSPF Areas and Hierarchy

Flooding an entire large AS can generate excessive traffic.
OSPF therefore divides an AS into:

Areas

- Each area is a smaller flooding domain.
- Routers maintain link-state information for routing.
- One area acts as the **backbone area**.
- Backbone area connects the other areas.
- Backbone area identification = **0**.

Thus OSPF uses two levels of hierarchy:

Autonomous System → Area

This reduces the flooding scope and supports larger networks.

Figure 4.75 – OSPF Areas

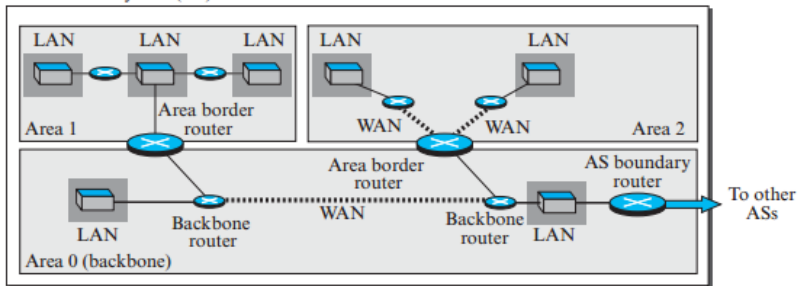
Figure 4.75 *Forwarding tables in OSPF*

Forwarding table for R1			Forwarding table for R2			Forwarding table for R3		
Destination network	Next router	Cost	Destination network	Next router	Cost	Destination network	Next router	Cost
N1	—	4	N1	R1	9	N1	R2	12
N2	—	5	N2	—	5	N2	R2	8
N3	R2	8	N3	—	3	N3	—	3
N4	R2	12	N4	R3	7	N4	—	4

Figure 4.76 – OSPF Area Structure

Figure 4.76 *Areas in an autonomous system*

Autonomous System (AS)



OSPF Link-State Advertisements

OSPF uses five types of link-state advertisements:

- ① **Router link**
- ② **Network link**
- ③ **Summary link to network**
- ④ **Summary link to AS border router**
- ⑤ **External link**

Purpose

These advertisements describe routers, networks, area information and networks outside the AS.

OSPF implementation

- Operates as a network-layer program.
- Uses IP for message propagation.
- IP protocol field = **89**.
- Most implementations use OSPF version 2.

Figure 4.77 – OSPF Link-State Advertisements

Figure 4.77 Five different LSPs

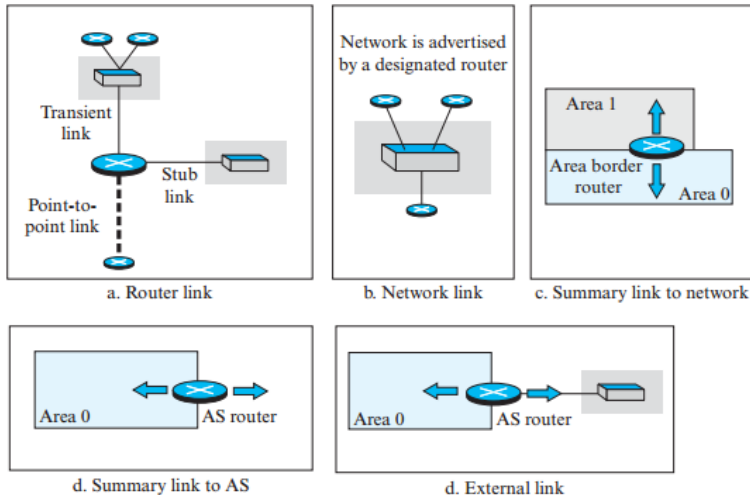
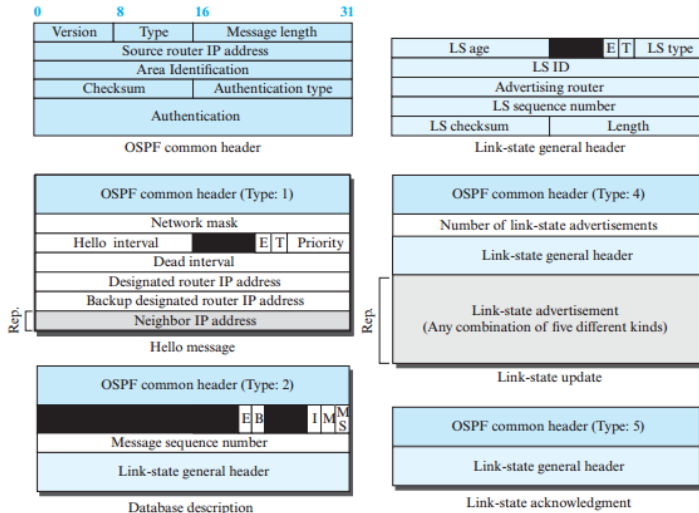


Figure 4.78 – OSPF LSA Information

Figure 4.78 OSPF message formats



OSPF Messages and Performance

Five OSPF message types:

- ① **Hello** – discovers and maintains neighbors
- ② **Database Description** – obtains LSDB information
- ③ **Link-State Request** – requests specific LS information
- ④ **Link-State Update** – carries link-state information
- ⑤ **Link-State Acknowledgment** – provides reliability

Authentication

The OSPF common header provides authentication capability.

Performance

- LSP flooding can consume bandwidth.
- Convergence is generally quick after flooding.
- Each router runs Dijkstra's algorithm.
- Completed LSDB makes routers relatively independent.

BGP4 – Border Gateway Protocol

BGP4 is the Internet's **interdomain routing protocol**.

It is based on the **path-vector algorithm** and provides information about reachability of networks.

Example Internet

An Internet may contain several ASs.

Each AS uses an intradomain protocol such as:

RIP or OSPF

BGP connects the ASs.

Two BGP variations:

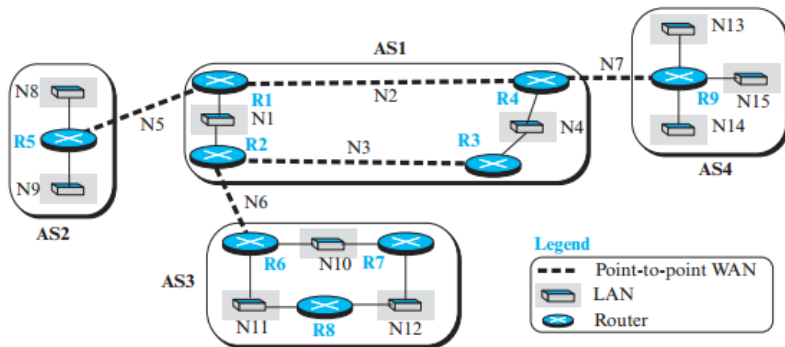
- **eBGP** – between different autonomous systems
- **iBGP** – within an autonomous system

BGP uses:

TCP port 179

Figure 4.79 – Autonomous Systems

Figure 4.79 *A sample internet with four ASs*



eBGP and iBGP

External BGP – eBGP

Border routers belonging to different ASs establish BGP sessions. The routers exchange reachability information for networks in their respective ASs.

Internal BGP – iBGP

Routers within the same AS exchange the interdomain reachability information learned through BGP.

For an AS with n routers, a fully connected iBGP arrangement has:

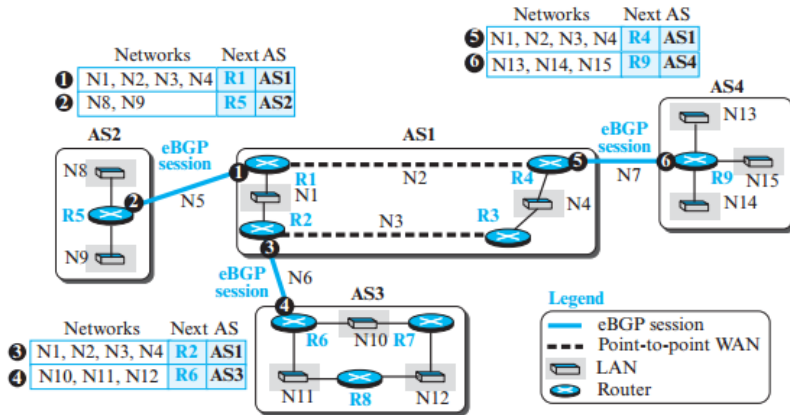
$$\frac{n(n-1)}{2}$$

sessions.

BGP sessions are logical TCP connections and may span multiple physical networks.

Figure 4.80 – eBGP Routing Information

Figure 4.80 eBGP operation



BGP – Injection into Intradomain Routing

BGP collects interdomain reachability information.
This information is then injected into intradomain forwarding tables.

Stub AS

A default route can be added through the border router.

Other networks → BGP border router

Transient AS

The path information may need to be injected more extensively into the intradomain forwarding tables.

The cost of reaching foreign networks can be related to the cost of reaching the first AS in the path.

Figure 4.82 – BGP Routing Information

Figure 4.82 *Finalized BGP path tables*

Networks	Next	Path	Networks	Next	Path	Networks	Next	Path
N8, N9	R5	AS1, AS2	N8, N9	R1	AS1, AS2	N8, N9	R2	AS1, AS2
N10, N11, N12	R2	AS1, AS3	N10, N11, N12	R6	AS1, AS3	N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4	N13, N14, N15	R1	AS1, AS4	N13, N14, N15	R4	AS1, AS4
Path table for R1			Path table for R2			Path table for R3		
Networks	Next	Path	Networks	Next	Path	Networks	Next	Path
N8, N9	R1	AS1, AS2	N1, N2, N3, N4	R1	AS2, AS1	N1, N2, N3, N4	R2	AS3, AS1
N10, N11, N12	R1	AS1, AS3	N10, N11, N12	R1	AS2, AS1, AS3	N8, N9	R2	AS3, AS1, AS2
N13, N14, N15	R9	AS1, AS4	N13, N14, N15	R1	AS2, AS1, AS4	N13, N14, N15	R2	AS3, AS1, AS4
Path table for R4			Path table for R5			Path table for R6		
Networks	Next	Path	Networks	Next	Path	Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1	N1, N2, N3, N4	R6	AS3, AS1	N1, N2, N3, N4	R4	AS4, AS1
N8, N9	R6	AS3, AS1, AS2	N8, N9	R6	AS3, AS1, AS2	N8, N9	R4	AS4, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4	N13, N14, N15	R6	AS3, AS1, AS4	N10, N11, N12	R4	AS4, AS1, AS3
Path table for R7			Path table for R8			Path table for R9		

Figure 4.83 – Interdomain Forwarding Tables

Figure 4.83 Forwarding tables after injection from BGP

Des.	Next	Cost
N1	—	1
N4	R4	2
N8	R5	1
N9	R5	1
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R1

Des.	Next	Cost
N1	—	1
N4	R3	2
N8	R1	2
N9	R1	2
N10	R6	1
N11	R6	1
N12	R6	1
N13	R3	3
N14	R3	3
N15	R3	3

Table for R2

Des.	Next	Cost
N1	R2	2
N4	—	1
N8	R2	3
N9	R2	3
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R3

Des.	Next	Cost
N1	R1	2
N4	—	1
N8	R1	2
N9	R1	2
N10	R3	3
N11	R3	3
N12	R3	3
N13	R9	1
N14	R9	1
N15	R9	1

Table for R4

Des.	Next	Cost
N8	—	1
N9	—	1
0	R1	1

Table for R5

Des.	Next	Cost
N10	—	1
N11	—	1
N12	R7	2
0	R2	1

Table for R6

Des.	Next	Cost
N10	—	1
N11	R6	2
N12	—	1
0	R6	2

Table for R7

Des.	Next	Cost
N10	R6	2
N11	—	1
N12	—	1
0	R6	2

Table for R8

Des.	Next	Cost
N13	—	1
N14	—	1
N15	—	1
0	R4	1

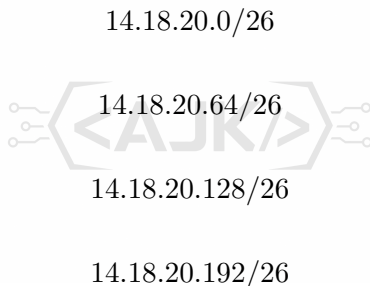
Table for R9

BGP Address Aggregation

Global forwarding tables can become extremely large.

BGP therefore supports **prefix aggregation**.

Example:



can be aggregated as:

14.18.20.0/24

provided all four prefixes can be reached through the same path.

BGP Path Attributes

BGP routes can contain up to seven path attributes.

Attribute	Type	Purpose
ORIGIN	Well-known mandatory	Source of routing information
AS-PATH	Well-known mandatory	AS sequence; loop prevention
NEXT-HOP	Well-known mandatory	Next router
MULTI-EXIT-DISC	Optional non-transitive	Select among exit paths
LOCAL-PREF	Well-known discretionary	Administrative preference
ATOMIC-AGGREGATE	Well-known discretionary	Indicates non-aggregated prefix
AGGREGATOR	Optional transitive	Identifies aggregation information

Attributes are classified as:

Well-known / Optional

and further as:

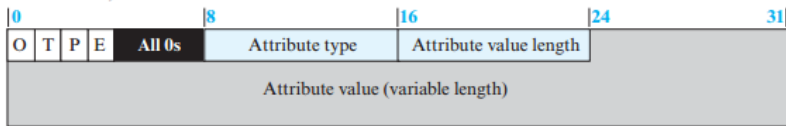
Mandatory / Discretionary / Transitive / Intransitive

Figure 4.84 – BGP Path Attributes

Figure 4.84 *Format of path attribute*

O: Optional bit (set if attribute is optional)
P: Partial bit (set if an optional attribute is lost in transit)

T: Transitive bit (set if attribute is transitive)
E: Extended bit (set if attribute length is two bytes)



BGP Route Selection

When multiple routes to a destination are available, BGP must select one route.

Unlike RIP or OSPF, selection is not based simply on the shortest-path tree.

The route contains several attributes.

Important

The **LOCAL-PREF** attribute is related to the policy imposed by the administration.

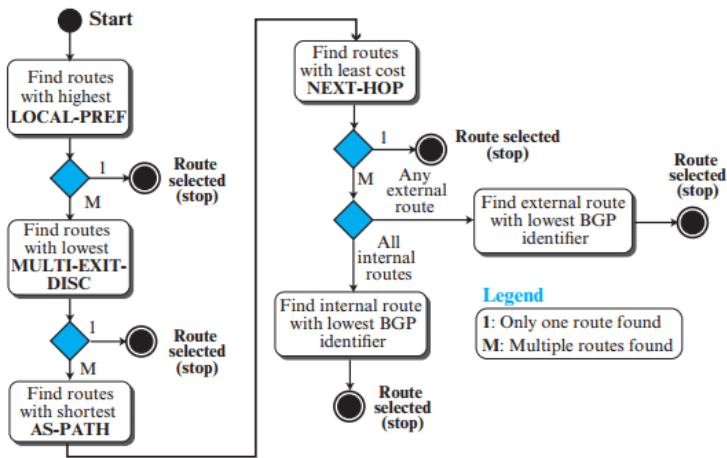
The selection process examines routes step by step.

- Routes satisfying each criterion are retained.
- If one route remains, it is selected.
- Otherwise the next criterion is applied.

Thus BGP route selection can incorporate administrative routing policies.

Figure 4.85 – BGP Route Selection

Figure 4.85 Flow diagram for route selection



BGP Messages

BGP uses four message types:

1. OPEN

Establishes a neighborhood relationship after creating a TCP connection.

2. UPDATE

The main BGP message.

Used to:

- Withdraw previously advertised destinations
- Advertise new destinations And Perform both operations

3. KEEPALIVE

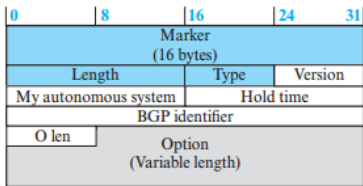
Exchanged periodically to indicate that the peer is still alive.

4. NOTIFICATION

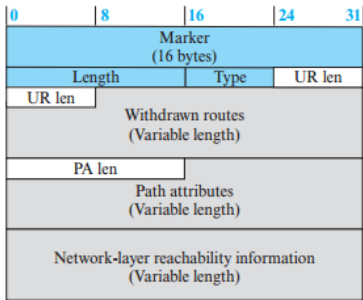
Sent when an error occurs or when a router wants to close the session.

Figure 4.86 – BGP Message Format

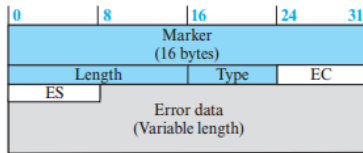
Figure 4.86 BGP messages



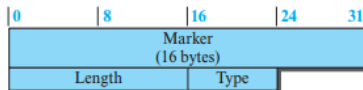
Open message (type 1)



Update message (type 2)



Notification message (type 3)



Keepalive message (type 4)

Fields in common header

Marker: Reserved for authentication

Length: Length of total message in bytes

Type: Type of message (1 to 4)

Abbreviations

O len: Option length

EC: Error code

ES: Error subcode

UR len: Unfeasible route length

PA len: Path attribute length

Routing Algorithms – Quick Comparison

Algorithm	Main Idea	Core Mechanism
Distance Vector	Exchange distance information with neighbors	Bellman–Ford
Link State	Build a complete network map	LSDB + Dijkstra
Path Vector	Exchange complete paths and apply policy	Path information + policy

Remember

DV → Bellman–Ford LS → Dijkstra PV → Policy

RIP vs. OSPF vs. BGP

	RIP	OSPF	BGP4
Algorithm	Distance Vector	Link State	Path Vector
Domain	Intra-AS	Intra-AS	Inter-AS
Main Metric	Hop count	Configurable link cost	Path attributes / policy
Core Algorithm	Bellman-Ford	Dijkstra	Policy-based path selection
Transport / Protocol	UDP 520	IP protocol 89	TCP 179
Hierarchy	AS	AS + Areas	Inter-AS
Main Strength	Simple	Detailed network knowledge	Policy and reachability